

[← Go Back](#)

Hardware

MKR WiFi 1010

Tutorials

Debugging SAM-Based
Arduino® Boards with
Atmel-ICE

Accessing the Built-in RGB
LED on the MKR WiFi 1010

How to Connect Sensors to
the MKR WiFi 1010

Connecting MKR WiFi 1010
to a Wi-Fi Network

MKR WiFi 1010 Bluetooth®
Low Energy

Host a Web Server on the
MKR WiFi 1010

MKR WiFi 1010 Battery
Application Note

**Using the Segger J-Link
Debugger with the MKR
Boards**

Sending Data over MQTT

Serial to OLED with MKR
WiFi 1010

Powering MKR WiFi 1010
with Batteries

RTC (Real Time Clock) with
MKR WiFi 1010 and OLED
Display

Scanning Networks with
MKR WiFi 1010

Securely Connecting an
Arduino MKR WiFi 1010 to
AWS IoT Core

Web Server Access Point
(AP) Mode with MKR WiFi
1010

Home / Hardware / MKR WiFi 1010 /

Using the Segger J-Link Debugger with the MKR Boards

Using the Segger J-Link Debugger with the MKR Boards

This tutorial teaches you how to set up a MKR board with the Segger J-link debugger.

Author · Benjamin Dannegård · Last revision · 03/03/2024

ON THIS PAGE

Overview

Required Hardware and
Software

J-Link Debug Probes

The SWD Interface

Instructions

Conclusion

—

+

+

Overview

This tutorial will show you how to debug an Arduino sketch using an Arduino MKR board and the Segger J-Link probe. It will go through how to connect these and use the Arduino IDE 2 to then debug a sketch.

Required Hardware and Software

Arduino IDE version 2.3.0 or higher

Segger J-link device (**EDU** or **BASE**)

Arduino MKR WiFi 1010 (other boards from the MKR family works as well).

Soldering equipment

MKR Proto shield

[Help](#)

J-Link Debug Probes

programming probes in the market. They have provide solid value to embedded development for over a decade. J-Link supports a wide range of CPUs and architectures, everything from single 8051 to mass market Cortex-M to high-end cores like Cortex-A. One of the main features of J-Link debug probes is its support of several debugging/programming interfaces like JTAG, SWD and SWO. For debugging SAM-based Arduino® boards with A J-Link probe, you are going to use its **SWD interface**.

The SWD Interface

JTAG is the de facto interface for debugging and accessing a processor registers. This interface has been used for many years, and it is still largely used today. But JTAG has a drawback: **it uses many signals (or pins), at least four**. To address this drawback, ARM® created an alternative debug interface called the Serial Wire Debug (SWD) interface.

The **SWD interface** is a **two-pin** (SWDIO/SWCLK) electrical alternative to the JTAG interface that has the same JTAG protocol on top. The SWD interface can be used to program the firmware of a processor and access its registers for debugging purposes. This interface and its associated protocol are now available in nearly all ARM® Cortex®-A, Cortex®-R, and Cortex®-M processors.

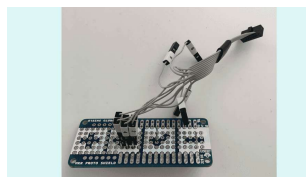
Instructions

Before you can connect your J-Link probe to your Arduino® MKR board, you must prepare the SWD interface pins of the board. The Arduino® MKR boards have the SWD interface pins, SWDIO and SWCLK, connected to the **J2 header**. The J2 header of the Arduino® MKR boards is located on the underside of the board in the exposed pads.

Note: The goal is to reach the exposed pads that are located on the underside of the board, feel free to choose one of the two methods that are explained in this tutorial or to implement another solution not mentioned here.

Connecting the Board and Device with MKR Proto Shield

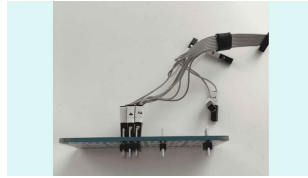
A method for connecting that doesn't require any soldering, uses a MKR Proto Shield and 6 2.54mm straight male headers to access the exposed pads of the MKR board as shown in the images below:



A MKR Proto Shield and 6 2.54mm straight male headers used to access the exposed pads of the board.

Besides not requiring any soldering, with this method the MKR Proto Shield remains fully reusable. Now

cable to the straight headers in the shield.



A MKR Proto Shield and 6 2.54mm straight male headers used to access the exposed pads of the board.

Connecting the Board and Device with Soldering

It is also possible to solder connectors onto the exposed pads on the back of the MKR board, that will then allow us to connect the J-Link adapter to the board. Refer to the illustration below to see where the connectors need to be soldered.

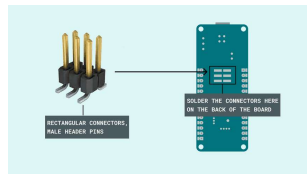


Illustration of soldering required

If there is an adapter or cable already plugged into the J-link, remove the FLAT cable or J Link adapter and expose the male pins. Connect the MKR to the J-link using female to female jumper wires, use the illustration below to know which pins to connect.

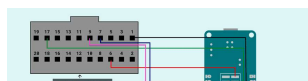


Illustration of connection

When the connections have been made, simply power the J-link probe and the MKR via USB (micro USB and type A cables).

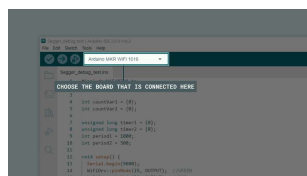
Installing the J-Link GDB Server Software

If you haven't already, download and install the [Arduino IDE 2](#), this will be used to debug our sketch. You will also need to download the J-Link GDB Server software, [here](#). The J-Link GDB Server is a remote server for the GNU Debugger (GDB) which allows to use a J-Link probe with GDB or any toolchain which uses GDB as debugging interface, like the Arduino IDE 2.

Note: write down where you installed the J-Link GDB Server software in your computer, this install path will be useful in the next step.

Debugging in Arduino IDE 2

Now we are ready to start debugging our sketch. Connect the power to the MKR board and the J-link to power them up. Start Arduino IDE 2 and select your MKR board in the deployable menu where you specify the board and port.



Then create or open the sketch that you want to debug. If you don't already have a sketch in mind, feel free to use the example sketch found at the end of this tutorial.

Select **Tools > Programmer >**

Segger J-Link from the Arduino IDE menus.

Now go to the folder where the sketch is located. Add a `.json` file in the same folder as your sketch and name it `debug_custom.json`.

The easiest way would be to create a text file and rename it

`debug_custom.json`. In the `.json` file, add the following lines:

```
1  [  
2    {  
3      "configId": "arduino  
4      "server": "jlink",  
5      "servertime": "jlink  
6      "device": "ATSAMD21G  
7      "interface": "SWD",  
8      "serverPath": "C:/Pr  
9      "serverpath": "C:/Pr  
10   }  
11 ]
```

If you are using a board other than the **MKR WiFi 1010**, you will need to adjust the

`arduino:samd:mkrwifi1010` part of the `configId` field. Hover the mouse pointer over the board selection in the Arduino IDE toolbar in order to learn the correct identifier for the board you are using.

The `"serverpath"` field needs to be set to the path of the "J-Link GDB Server CL" tool executable file that is

package you installed in the previous step. The file is named:

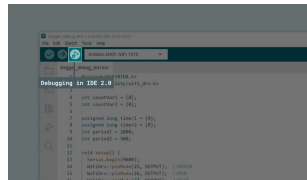
If you are using Windows:

JLinkGDBServerCL.exe

If you are using Linux or macOS:

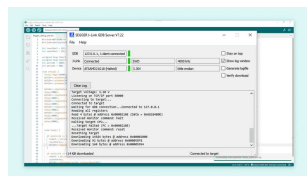
JLinkGDBServer

When you have finished creating the `debug_custom.json` file, click on the debugging icon.



Start debug feature in
Arduino IDE 2

You should see a window similar to the one shown below. Do not close this window, just minimize it and you are now ready to start debugging. You can add breakpoints, inspect variables, halt the execution and more.



J-link debug window in
Arduino IDE 2

Conclusion

In this tutorial you learned how to connect your MKR board to a Segger J-Link probe. And then how to use this setup to debug a sketch with Arduino IDE 2.

Now that you have your hardware

Debugging with the [Arduino IDE 2 tutorial](#). This tutorial goes through some key features of the Debugger, and includes pointers to get started.

Example Sketch

```
1  #include<WiFiNINA.h>
2  #include<utility/wifi_c
3
4  int countVar1 = {0};
5  int countVar2 = {0};
6
7  unsigned long timer1 =
8  unsigned long timer2 =
9  int period1 = 1000;
10 int period2 = 500;
11
12 void setup() {
13   Serial.begin(9600);
14   WiFiDrv::pinMode(25,
15   WiFiDrv::pinMode(26,
16   WiFiDrv::pinMode(27,
17
18   WiFiDrv::digitalWrite
19   delay(1000);
20   WiFiDrv::digitalWrite
21   delay(1000);
22   WiFiDrv::digitalWrite
23   delay(1000);
24   WiFiDrv::digitalWrite
25   delay(1000);
26   WiFiDrv::digitalWrite
27   delay(1000);
28   WiFiDrv::digitalWrite
```

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

wrong, you
can edit
this page
[here](#).

Was this article helpful?

